AXON.

Agentic eXecution & Oversight Network

Enterprise Multi-Agent Control Tower, Telemetry Observability & Governance Platform

1. Executive Summary

As organizations increasingly adopt multi-agent AI ecosystems, managing their execution paths, budgets, API token costs, and security compliance becomes a major challenge. **AXON. (Agentic eXecution & Oversight Network)** is an enterprise-grade AI Agent Control Tower serving as the central nervous system for multi-agent fleets. It addresses key operational challenges including runaway billing loops, lack of coordination visibility, and compliance risks of fully autonomous actions by offering:

- **Real-time Telemetry & Observability**: Visualizes agent task execution chains through a live Directed Acyclic Graph (DAG) Lineage Map and Step Replay Logs.
- **Cost Attribution & Guardrails**: Tracks LLM token spends per agent role and enforces daily spending limits.
- **Anomaly Interception**: Automatically detects infinite recursion loops, halting execution to prevent token drain.
- **Human-in-the-Loop Governance**: Integrates an approval queue to stage high-risk operations until approved by an operator.

2. Architecture & Techstack

AXON. is designed with a decoupled client-server architecture utilizing robust, modern frameworks to ensure real-time responsiveness and low latency:

Tier	Technology	Purpose
Frontend	React 18, Vite, Material UI (MUI)	High-performance client UI, responsive grid modules, dark/light theme options, and custom CSS styling.

Data Visualization	Recharts, Canvas-based DAG Graph	Renders live token spend graphs, historical budgets, and real-time communication flows between agents.
Telemetry Stream	SSE (Server-Sent Events)	Establishes a persistent uni-directional stream to push logs, task start/stop status, and alerts from backend to client.
Backend APIs	FastAPI (Python 3.12), Uvicorn	High-performance async endpoints, database transaction routing, and background thread execution.
Database Engine	SQLite & SQLAlchemy ORM	Relational schema database for storing settings, logs, costs, approvals, dependency configurations, and agent structures.
AI Cognition	Google Gemini Pro/Flash API	Powers the real-time AI assistant (RELAY) and drives mock specialist simulations.

3. Database Structure & Schema

AXON. utilizes an SQLite database (located at `backend/control_tower.db`) with structured relations to maintain absolute referential integrity. Below is the relational mapping of each database entity:

3.1 Table: `agents`

Stores definition profiles, models, and spending constraints for each agent type.

Column	Type	Constraints	Description
id	VARCHAR	Primary Key	Unique ID of the agent (e.g. 'finance_agent', 'devops_agent').
name	VARCHAR	Not Null	Descriptive name of the agent.
role	VARCHAR	Not Null	Instructions defining what the agent does.
status	VARCHAR	Default 'ACTIVE'	Current state: 'ACTIVE', 'SUSPENDED', or 'IDLE'.
cost_limit	FLOAT	Not Null	Spending limit (USD) before automatic lock is triggered.
configuration	VARCHAR (JSON)	-	JSON parameters for the LLM (e.g. model, temperature).

3.2 Table: `tasks`

Logs all orchestrator queries and specialist sub-task runs.

Column	Type	Constraints	Description

id	VARCHAR	Primary Key	Unique UUID for the task execution.
agent_id	VARCHAR	Foreign Key (agents.id)	Attributes the execution to a specific agent.
status	VARCHAR	Not Null	Current run state: 'RUNNING', 'SUCCESS', 'FAILED', or 'PENDING_APPROVAL'.
description	VARCHAR	-	User prompt or sub-task objective description.
created_at	DATETIME	Default utcnow	Timestamp when the run was initiated.
completed_at	DATETIME	-	Timestamp when the run finished.

3.3 Table: `logs`

Logs granular events generated by the agents during task execution.

Column	Type	Constraints	Description
id	INTEGER	Primary Key Autoincrement	Unique log identifier.
task_id	VARCHAR	Foreign Key (tasks.id)	Links the log statement to its parent execution thread.
level	VARCHAR	Not Null	Severity level: 'INFO', 'WARNING', or 'ERROR'.
message	VARCHAR	Not Null	The text log details.

timestamp	DATETIME	Default utcnow	Granular event timestamp.

3.4 Table: `costs`

Tracks financial expenditures and API token consumption metrics per execution.

Column	Type	Constraints	Description
id	INTEGER	Primary Key Autoincrement	Unique cost entry ID.
task_id	VARCHAR	Foreign Key (tasks.id)	Links token usage to the specific task execution.
prompt_tokens	INTEGER	Not Null	Input tokens consumed.
completion_tokens	INTEGER	Not Null	Output tokens generated.
cost_usd	FLOAT	Not Null	Dollar spend (USD) calculated from model pricing.
timestamp	DATETIME	Default utcnow	Timestamp when the cost was registered.

3.5 Table: `approvals`

Holds governance tasks waiting for human intervention.

Column	Type	Constraints	Description
id	VARCHAR	Primary Key	Unique approval request ID.
task_id	VARCHAR	Foreign Key (tasks.id)	Links the approval request to the suspended task.

action_name	VARCHAR	Not Null	The action name needing approval (e.g. 'Server Provisioning').
status	VARCHAR	Default 'PENDING'	Governance status: 'PENDING', 'APPROVED', or 'REJECTED'.
reason	VARCHAR	-	Operator comments justifying the approval decision.
reviewer	VARCHAR	-	The operator user ID executing the review (e.g. 'Admin').
created_at	DATETIME	Default utcnow	Timestamp when the request entered the queue.
updated_at	DATETIME	-	Timestamp when the decision was finalized.

3.6 Table: `alerts`

Stores anomalies (loops, budget breaches) caught by the monitoring thread.

Column	Type	Constraints	Description
id	INTEGER	Primary Key Autoincrement	Unique alert ID.
task_id	VARCHAR	Foreign Key (tasks.id)	Links the alert to the offending task thread.
type	VARCHAR	Not Null	Type of anomaly: 'LOOP_DETECTION' or 'COST_LIMIT'.

severity	VARCHAR	Not Null	Severity rating: 'CRITICAL' or 'WARNING'.
description	VARCHAR	Not Null	Explanation of the anomaly.
resolved	BOOLEAN	Default False	Indicates if the administrator has cleared the alert.
timestamp	DATETIME	Default utcnow	Timestamp of the event.

4. Operations & Governance Guidelines

4.1 Infinite Loop Breaker

When an agent is executing a sub-task, the system monitors log consistency. If an agent emits identical warning messages or queries recursively (more than 3 times), AXON. instantly halts the task, marks the execution status as `FAILED`, registers a critical warning log, and files a `LOOP_DETECTION` alert. This shields the company from token leakage during service outages.

4.2 Human-in-the-Loop Approval Queue

High-risk transactions or tasks requesting cost resource margins above budget limits trigger an automatic hold state. The task's execution freezes, its state shifts to `PENDING_APPROVAL`, and a request record is queued inside the approvals dashboard. Developers or administrators review the context and click "Approve" or "Reject". On approval, the task is unlocked and completes successfully.

4.3 System Settings Console

Unlocked via credentials (`admin` / `admin123`) in the top-right Settings Drawer, this console allows operators to configure execution speed delays, loop breaker caps, and cost caps globally. Additionally, administrators can perform a hard database refresh, purging telemetry history and reinstating default seed agents with a single click.

5. Installation & Execution Guide

5.1 Prerequisites

- Python 3.12+ installed
- Node.js (v18+) and npm installed
- Git CLI tools

5.2 Backend Installation

Navigate to the backend directory, install requirements, and run the FastAPI server:

```
cd backend pip install -r requirements.txt python seed_data.py python main.py
```

Note: The backend server starts by default on `http://127.0.0.1:8000`. The database will automatically initialize and populate with the default agents.

5.3 Frontend Installation

Navigate to the frontend directory, install npm packages, and spin up the Vite development server:

```
cd frontend npm install npm run dev
```

Note: The frontend development server will launch on `http://localhost:5173` (or the next available port). Open this URL in your web browser to explore AXON.

6. Verification Checklists

Ensure the following steps are verified during demonstrations:

1. **Engine Runs**: Input prompts in the console and watch the logs stream in.
2. **Stop Button**: Click "Stop Engine" during execution to verify instant thread cancellation.
3. **Governance**: Select the "Finance Approval Request" scenario, then approve the pending item inside the approvals center.
4. **Loop Anomaly**: Run the "Retry Loop Anomaly" scenario and watch the alert feed register the loop breach.